

[Vai al contenuto](#)

DISPENSA DELLA LEZIONE

React per interfacce dinamiche

Perché nasce React, come installarlo con Vite tramite pnpm o npm e come gestire liste dinamiche in JSX.



CAPITOLO 1

Il problema che React risolve

NOTA

React è una libreria JavaScript pensata per costruire interfacce utente, cioè tutto ciò che l'utente vede e con cui interagisce in una pagina web.

React nasce per rendere più semplice la costruzione di interfacce complesse, soprattutto quando la pagina deve cambiare spesso in base ai dati. Senza uno strumento come React, aggiornare tanti elementi a mano nel DOM diventa ripetitivo, fragile e difficile da mantenere.

Con React l'interfaccia viene costruita come insieme di componenti, cioè blocchi riutilizzabili che descrivono una parte della pagina. Questo aiuta a separare meglio la struttura visiva dalla logica e riduce la quantità di codice duplicato.

React è utile per gestire il layout in componenti più piccoli e per riusarli più volte con dati diversi. Questo aiuta a semplificare la manutenzione del codice e a ridurre il rischio di errori.

CAPITOLO 2

Vite: lo strumento per creare progetti React

NOTA

Vite è uno strumento che prepara automaticamente tutta la struttura di file e configurazioni necessarie per far funzionare un progetto React, senza doverla scrivere a mano.

Scrivere un progetto React da zero richiederebbe di configurare manualmente molti file, come quelli che gestiscono la traduzione del JSX in JavaScript puro o l'aggiornamento automatico della pagina durante lo sviluppo. Vite si occupa di tutto questo lavoro iniziale, così ci si può concentrare direttamente sulla scrittura dei componenti.

Un altro motivo per cui si usa Vite è la velocità: quando si modifica un file del progetto, Vite aggiorna la pagina nel browser in pochissimo tempo, senza dover ricaricare tutto da capo. Questo rende molto più comodo lavorare mentre si scrive codice React.

Vite si avvia tramite un comando che lancia un wizard, cioè una piccola procedura guidata che fa qualche domanda su come impostare il progetto, prima di generare i file necessari.

CAPITOLO 3

Installare React con pnpm o npm

NOTA

pnpm e **npm** sono entrambi package manager, cioè programmi che scaricano e gestiscono le librerie del progetto. Il risultato finale del progetto React è identico: cambia solo il comando da digitare nel terminale.

La differenza principale è che **pnpm** salva i pacchetti una sola volta sul disco e li collega ai progetti, mentre **npm** li copia interamente in ogni progetto. Per questo **pnpm** è generalmente più rapido e occupa meno spazio, ma il codice React che si scrive non cambia in base alla scelta.

Per creare un nuovo progetto, basta aprire il terminale nel percorso dove si vuole lavorare ed avviare il comando **pnpm create vite** e rispondere alle domande del wizard che compare nel terminale.

Esempio

```
# Con pnpm
pnpm create vite

# Con npm
npm create vite@latest
```

Il wizard chiede alcune informazioni in sequenza. Per il nome del progetto si può digitare un punto **.**, così Vite usa direttamente la cartella corrente invece di crearne una nuova. Alle domande successive si seleziona il framework **React** e come variante **JavaScript**.

Dopo aver risposto alle domande del wizard, si installano le dipendenze indicate nel file **package.json** e si avvia il server di sviluppo.

```
# Avvio del server di sviluppo
pnpm dev
npm run dev
```

Con npm serve scrivere **run** prima del nome dello script, mentre con pnpm il comando **run** è opzionale e si può omettere.

CAPITOLO 4

I componenti in React

NOTA

Un componente è una funzione JavaScript che restituisce del JSX, cioè descrive una porzione di interfaccia. Il nome di un componente inizia sempre con la lettera maiuscola, per distinguerlo dai tag HTML normali.

Un componente serve a racchiudere in un solo posto la struttura di una parte di pagina, così da poterla riutilizzare ovunque sia necessaria senza riscriverla. Ogni componente viene definito una sola volta e può essere richiamato più volte, anche con dati diversi.

Per usare un componente in un altro file, bisogna prima esportarlo con `export default` alla fine del file in cui è definito. Nel file che deve mostrarlo, si usa poi `import` per portarlo dentro e si scrive il suo nome come se fosse un tag, ad esempio `<NomeComponente />`.

Il percorso indicato in `import` deve corrispondere alla posizione reale del file, contando dalla cartella del file che sta importando.

Esempio

`src/components/Saluto.jsx`

```
function Saluto() {  
  return (  
    <p>Benvenuto nel corso di React</p>  
  );  
}  
  
export default Saluto;
```

`src/App.jsx`

```
import Saluto from './components/Saluto.jsx';

function App() {
  return <Saluto />;
}

export default App;
```

CAPITOLO 5

JSX e blocchi riutilizzabili

NOTA

JSX non è HTML: è una sintassi che viene trasformata in chiamate JavaScript prima di arrivare al browser, quindi segue alcune regole diverse dall'HTML classico.

JSX permette di scrivere strutture simili all'HTML dentro il JavaScript, per descrivere ciò che deve comparire a schermo. Ogni blocco JSX deve avere un solo elemento principale, che può contenere altri elementi al suo interno.

Per mostrare il valore di una variabile dentro il testo o dentro un attributo, si usano le parentesi graffe `{ }`. Dentro le graffe si può scrivere qualsiasi espressione che restituisce un valore, come una variabile o un calcolo; quando quel valore cambia, React aggiorna automaticamente solo quella parte, senza riscrivere il resto della struttura.

Differenze principali rispetto all'HTML

JSX assomiglia molto all'HTML, ma alcune regole di scrittura cambiano perché il codice deve essere trasformato in JavaScript valido.

- `class` diventa `className`, perché `class` è una parola riservata in JavaScript.
- `for` (usato sul tag `label`) diventa `htmlFor`, per lo stesso motivo.
- Ogni tag deve essere sempre chiuso, anche quelli senza contenuto, come `<img />` o `<hr />`.
- L'attributo `style` non è una stringa ma un oggetto JavaScript; le proprietà semplici come `color` restano invariate, mentre quelle composte da più parole, come `background-color` o `max-width`, si scrivono in camelCase senza trattino, cioè `backgroundColor` e `maxWidth`.
- Gli eventi si scrivono in camelCase, ad esempio `onClick` invece di `onclick`.

Esempio

```
src/components/Titolo.jsx
```

```
function Titolo() {  
  const nomeCorso = 'React Base';  
  const stile = { color: 'darkblue', backgroundColor: 'lightgray' };  
  
  return (  
    <section className="intro-corso">  
      <h1 style={stile}>{nomeCorso}</h1>  
      <p>Interfacce dinamiche con componenti.</p>  
      <hr />  
    </section>  
  );  
}  
  
export default Titolo;
```

src/App.jsx

```
import Titolo from './components/Titolo.jsx';  
  
function App() {  
  return <Titolo />;  
}  
  
export default App;
```

CAPITOLO 6

I Fragment in JSX

NOTA

Un Fragment è un contenitore invisibile: raggruppa più elementi JSX senza aggiungere un tag reale nella pagina HTML finale.

Come già visto, ogni componente JSX deve restituire un solo elemento principale. Quando gli elementi da restituire sono più di uno e non serve un tag come `div` o `section` per contenerli, si può usare un Fragment al posto di un tag qualsiasi.

Il Fragment si scrive con la sintassi `<>...</>`, senza nome e senza attributi. Serve esclusivamente a soddisfare la regola del singolo elemento principale, senza produrre un tag visibile nell'HTML generato.

Usare un Fragment è utile quando aggiungere un `div` in più creerebbe una struttura HTML non necessaria, ad esempio dentro una lista dove ogni elemento deve restare allo stesso livello senza contenitori aggiuntivi.

Esempio

`src/components/Info.jsx`

```
function Info() {  
  return (  
    <>  
      <h2>Informazioni corso</h2>  
      <p>Il corso copre le basi di React.</p>  
    </>  
  );  
}  
  
export default Info;
```

`src/App.jsx`

```
import Info from './components/Info.jsx';

function App() {
  return <Info />;
}

export default App;
```

CAPITOLO 7

Iterazioni in JSX con map

NOTA

`map` è un metodo degli array già usato in JavaScript: qui viene applicato per trasformare i dati in blocchi JSX, invece di scrivere ogni elemento a mano.

Quando si ha una lista di elementi, il metodo `map` permette di trasformare ogni elemento dell'array in un blocco JSX. In questo modo si evita di copiare e incollare più volte lo stesso codice statico.

Il vantaggio principale è che la lista diventa dinamica: se i dati cambiano, cambia anche ciò che viene mostrato a schermo. Basta modificare l'array e l'interfaccia segue automaticamente il contenuto.

Il metodo `map` restituisce un nuovo array, quindi è perfetto per costruire una sequenza di elementi visivi a partire da dati già pronti.

Esempio

`src/components/ListaStudenti.jsx`

```
function ListaStudenti() {  
  const studenti = ['Anna', 'Luca', 'Sara'];  
  
  return (  
    <ul>  
      {studenti.map((studente) => (  
        <li>{studente}</li>  
      ))}  
    </ul>  
  );  
}  
  
export default ListaStudenti;
```

CAPITOLO 8

La key prop nelle liste

NOTA

La **key** non viene mai mostrata a schermo: serve solo a React internamente, per riconoscere ogni elemento della lista tra un aggiornamento e l'altro.

Quando una lista viene generata con **map**, ogni elemento deve avere una **key** unica. La **key** è un identificatore che aiuta React a riconoscere ogni elemento della lista in modo corretto.

Questo è importante perché, se la lista cambia, React deve capire quale elemento è stato aggiunto, rimosso o spostato. Senza una **key** adatta, il comportamento può diventare impreciso o meno efficiente.

La **key** deve essere stabile e diversa per ogni elemento della stessa lista. Quando esiste un identificatore univoco nei dati, è la scelta migliore.

Esempio

src/components/ListaCorsi.jsx

```
function ListaCorsi() {  
  const corsi = [  
    { id: 1, nome: 'HTML' },  
    { id: 2, nome: 'CSS' },  
    { id: 3, nome: 'JavaScript' }  
  ];  
  
  return (  
    <ul>  
      {corsi.map((corso) => (  
        <li key={corso.id}>{corso.nome}</li>  
      ))}  
    </ul>  
  );  
}  
  
export default ListaCorsi;
```